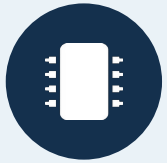
IoT: Estación Meteorológica

Productor y consumidor con Apache Kafka



Daniel Juárez · 23709
Alexander de la Cruz · 23735
CC3067 Redes · Universidad del Valle de Guatemala · 12 de noviembre de 2026

Arquitectura de la solución



Productor

producer.py · nodo de sensores simulado



Kafka (Edge Server)

161.35.11.141:9092 · topics 23709 y 23709_compact



Consumidor

consumer.py · gráfica en tiempo real

Autenticación

SASL_PLAINTEXT con mecanismo PLAIN, usuario y contraseña del laboratorio.

Mensajes

Key sensor1, un envío cada 15 a 30 segundos hasta que se interrumpe el productor.

Dos versiones

Primero JSON legible; después un payload de 3 bytes (24 bits) para redes restringidas.

Simulación de sensores

Sensor	Rango	Tipo	Generación
Temperatura (termómetro)	0 – 110.00 °C	Float, 2 decimales	Normal, media 55 y σ = 15
Humedad relativa (higrómetro)	0 – 100 %	Entero	Normal, media 55 y σ = 15
Dirección del viento	N, NO, O, SO, S, SE, E, NE	Categórico (8 valores)	Uniforme (random.choice)

```
def _normal(media=55, sigma=15, lo=0, hi=110):
    return min(hi, max(lo, random.gauss(media, sigma)))

def generar_lectura():
    return {
        "temperatura": round(_normal(), 2),
        "humedad": int(round(_normal(hi=100))),
        "direccion_viento": random.choice(DIRECCIONES),
    }
```

Ejemplo de lectura en JSON

```
{"temperatura": 56.32,
 "humedad": 51,
 "direccion_viento": "SO"}
```

La normal se recorta al rango válido; con σ = 15 casi nunca se recorta. Condensar la lectura en un JSON hace el código modular y menos frágil.

JSON/SOAP: capa OSI y beneficios

¿A qué capa del modelo OSI pertenece?

A la capa 6, Presentación. JSON y SOAP (XML) definen la sintaxis y la estructura con que se serializan los datos, para que emisor y receptor los interpreten igual sin importar su plataforma o lenguaje.

Viajan sobre la capa 7: SOAP además es un protocolo de mensajería que suele ir sobre HTTP. En este laboratorio el JSON va dentro de un mensaje de Kafka, que usa TCP en la capa 4.

¿Qué beneficios tiene usar JSON/SOAP?

- Independiente del lenguaje y la plataforma
- Autodescriptivo: cada valor lleva el nombre de su campo
- Extensible: se agregan sensores sin romper al consumidor
- Fácil de leer, depurar y validar
- Estándar con librerías en casi todos los lenguajes

Costo: más bytes por mensaje. Aquí, 62–63 bytes en JSON contra 3 bytes en el formato compacto.

Envío de datos al Edge

Cómo funciona el productor

- Topic 23709, el carné de un integrante, para no cruzar datos con otras parejas
- Key sensor1: Kafka guarda en la misma partición los mensajes con la misma key
- Autenticación SASL_PLAINTEXT con PLAIN
- Envía cada 15 a 30 segundos (random.uniform) hasta Ctrl+C
- send().get() confirma la partición y el offset

```
producer = KafkaProducer(
    bootstrap_servers="161.35.11.141:9092",
    security_protocol="SASL_PLAINTEXT",
    sasl_mechanism="PLAIN",
    sasl_plain_username="estudiante",
    sasl_plain_password=...,
    key_serializer=str.encode)

while True:
    lectura = generar_lectura()
    valor = json.dumps(lectura).encode()
    producer.send(topic, key="sensor1",
                  value=valor)
    time.sleep(random.uniform(15, 30))
```

Evidencia: terminales (JSON)

Productor (python producer.py --modo json)

```
(.venv) gonzalomonzonch@gonzalos-ubuntu:~/Desktop/IOT_Lab$ python producer.py --modo json
/home/gonzalomonzonch/Desktop/IOT_Lab/producer.py:54: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  main()
Productor listo -> 161.35.11.141:9092 topic=23709 modo=json
[enviado] {'temperatura': 60.16, 'humedad': 73, 'direccion_viento': 'E'} (62 bytes) part=1 off=0
[enviado] {'temperatura': 42.1, 'humedad': 57, 'direccion_viento': 'NE'} (62 bytes) part=1 off=1
[enviado] {'temperatura': 66.16, 'humedad': 49, 'direccion_viento': 'SO'} (63 bytes) part=1 off=2
[enviado] {'temperatura': 63.36, 'humedad': 71, 'direccion_viento': 'E'} (62 bytes) part=1 off=3
[enviado] {'temperatura': 53.41, 'humedad': 94, 'direccion_viento': 'NE'} (63 bytes) part=1 off=4
[enviado] {'temperatura': 65.72, 'humedad': 46, 'direccion_viento': 'SO'} (63 bytes) part=1 off=5
[enviado] {'temperatura': 66.11, 'humedad': 51, 'direccion_viento': 'NE'} (63 bytes) part=1 off=6
[enviado] {'temperatura': 48.97, 'humedad': 50, 'direccion_viento': 'N'} (62 bytes) part=1 off=7
[enviado] {'temperatura': 34.11, 'humedad': 48, 'direccion_viento': 'N'} (62 bytes) part=1 off=8
[enviado] {'temperatura': 61.45, 'humedad': 29, 'direccion_viento': 'SE'} (63 bytes) part=1 off=9
[enviado] {'temperatura': 79.51, 'humedad': 49, 'direccion_viento': 'O'} (62 bytes) part=1 off=10
[enviado] {'temperatura': 33.93, 'humedad': 57, 'direccion_viento': 'E'} (62 bytes) part=1 off=11
[enviado] {'temperatura': 62.97, 'humedad': 75, 'direccion_viento': 'NO'} (63 bytes) part=1 off=12
[enviado] {'temperatura': 51.62, 'humedad': 81, 'direccion_viento': 'SO'} (63 bytes) part=1 off=13
```

Consumidor (python consumer.py --modo json)

```
(.venv) gonzalomonzonch@gonzalos-ubuntu:~/Desktop/IOT_Lab$ python consumer.py --modo json
Consumidor listo -> topic=23709 modo=json
[recibido] {'temperatura': 60.16, 'humedad': 73, 'direccion_viento': 'E'}
[recibido] {'temperatura': 42.1, 'humedad': 57, 'direccion_viento': 'NE'}
[recibido] {'temperatura': 66.16, 'humedad': 49, 'direccion_viento': 'SO'}
[recibido] {'temperatura': 63.36, 'humedad': 71, 'direccion_viento': 'E'}
[recibido] {'temperatura': 53.41, 'humedad': 94, 'direccion_viento': 'NE'}
[recibido] {'temperatura': 65.72, 'humedad': 46, 'direccion_viento': 'SO'}
[recibido] {'temperatura': 66.11, 'humedad': 51, 'direccion_viento': 'NE'}
[recibido] {'temperatura': 48.97, 'humedad': 50, 'direccion_viento': 'N'}
[recibido] {'temperatura': 34.11, 'humedad': 48, 'direccion_viento': 'N'}
[recibido] {'temperatura': 61.45, 'humedad': 29, 'direccion_viento': 'SE'}
[recibido] {'temperatura': 79.51, 'humedad': 49, 'direccion_viento': 'O'}
[recibido] {'temperatura': 33.93, 'humedad': 57, 'direccion_viento': 'E'}
[recibido] {'temperatura': 62.97, 'humedad': 75, 'direccion_viento': 'NO'}
[recibido] {'temperatura': 51.62, 'humedad': 81, 'direccion_viento': 'SO'}
[recibido] {'temperatura': 71.13, 'humedad': 46, 'direccion_viento': 'E'}
```

Misma partición

Todos los mensajes salen con part=1 porque comparten la key sensor1.

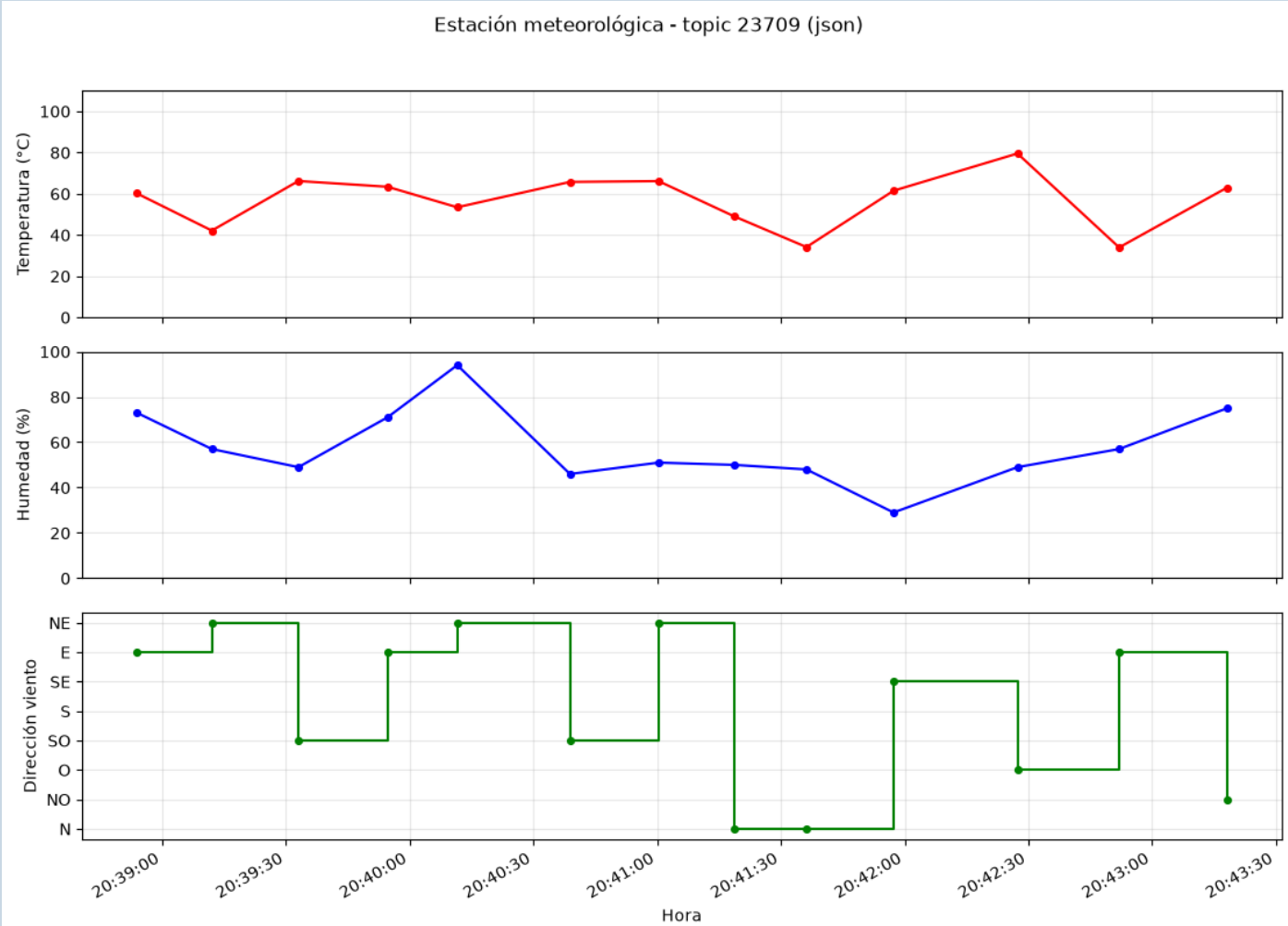
Offsets consecutivos

El productor registra los offsets 0 al 13, uno por lectura.

Mismos valores

El consumidor recibe exactamente las lecturas enviadas, en el mismo orden.

Gráfica en tiempo real (JSON)



Temperatura

Rango 0–110 °C, dato nuevo cada 15–30 s.

Humedad relativa

Rango 0–100 %, valores enteros.

Dirección del viento

Ocho categorías en el eje vertical.

El consumidor redibuja la gráfica con cada mensaje nuevo, incluyendo el historial del topic.

Pub/Sub de Kafka: pros y contras

Ventajas

- Desacopla: el productor y el consumidor no se conocen entre sí
- Escala con particiones y varios consumidores por topic
- Persiste los mensajes: se puede leer el historial desde un offset
- Tolera fallos replicando los topics entre brokers
- Mantiene el orden dentro de cada partición

Desventajas

- El broker es una pieza más que instalar, asegurar y operar
- El cliente es pesado y mantiene una conexión TCP: poco apto para dispositivos mínimos
- Agrega latencia y sobrecarga frente a un envío directo
- El orden solo se garantiza dentro de una partición
- Si el servidor Edge cae, se interrumpe el flujo

¿Cuándo usar Kafka?

Tiene sentido para

- Telemetría de muchos sensores o dispositivos a la vez
- Streaming y analítica en tiempo real
- Centralizar logs y métricas
- Comunicar microservicios mediante eventos
- Pipelines de datos que necesitan reprocesar el historial

No es la mejor opción para

- Dispositivos con poca memoria o energía, o redes como LoRa y Sigfox (mejor MQTT o CoAP)
- Peticiones y respuestas síncronas (REST, gRPC)
- Pocos mensajes o apps pequeñas: una cola simple o una base de datos bastan
- Transferir archivos grandes
- Control en tiempo real estricto

Payload de 3 bytes: mapa de bits



Temperatura: 14 bits

110.00 °C × 100 = 11000. Con 13 bits solo hay 8192 valores; con 14 bits hay 16384, y 11000 cabe.

Humedad: 7 bits

Entero de 0 a 100, son 101 valores. 2^7 = 128, así que 7 bits alcanzan.

Viento: 3 bits

Ocho direcciones = 2^3. N=0, NO=1, O=2, SO=3, S=4, SE=5, E=6, NE=7.

Encode y decode

```
# versión resumida de codec.py
def encode(l):
    temp = round(l["temperatura"] * 100)
    hum = int(l["humedad"])
    viento = DIRECCIONES.index(l["direccion_viento"])
    valor = (temp << 10) | (hum << 3) | viento
    return valor.to_bytes(3, "big")

def decode(datos):
    v = int.from_bytes(datos, "big")
    return {
        "temperatura": ((v >> 10) & 0x3FFF) / 100,
        "humedad": (v >> 3) & 0x7F,
        "direccion_viento": DIRECCIONES[v & 0x7],
    }
```

Ejemplo paso a paso

Lectura: 56.32 °C, 51 %, "SO"

```
56.32 × 100 = 5632
  → 010110000000000
51 → 0110011
SO → 3 → 011
```

Se concatenan 24 bits:
`01011000 00000001 10011011`

`Bytes enviados: 58 01 9B`

Evidencia: terminales (3 bytes)

Productor (python producer.py --modo compact)

```
(.venv) gonzalomonzonch@gonzalos-ubuntu:~/Desktop/IOT_Lab$ python producer.py --modo compact
/home/gonzalomonzonch/Desktop/IOT_Lab/producer.py:54: DeprecationWarning: key_serializer does not implement kafka.serializer.Serializer
  main()
Productor listo -> 161.35.11.141:9092 topic=23709_compact modo=compact
[enviado] {'temperatura': 60.66, 'humedad': 44, 'direccion_viento': 'E'} (3 bytes) part=1 off=0
[enviado] {'temperatura': 21.02, 'humedad': 52, 'direccion_viento': 'S'} (3 bytes) part=1 off=1
[enviado] {'temperatura': 75.73, 'humedad': 28, 'direccion_viento': 'O'} (3 bytes) part=1 off=2
[enviado] {'temperatura': 76.07, 'humedad': 64, 'direccion_viento': 'SO'} (3 bytes) part=1 off=3
[enviado] {'temperatura': 53.03, 'humedad': 15, 'direccion_viento': 'SO'} (3 bytes) part=1 off=4
[enviado] {'temperatura': 56.43, 'humedad': 75, 'direccion_viento': 'N'} (3 bytes) part=1 off=5
[enviado] {'temperatura': 40.92, 'humedad': 69, 'direccion_viento': 'O'} (3 bytes) part=1 off=6
[enviado] {'temperatura': 53.77, 'humedad': 60, 'direccion_viento': 'NO'} (3 bytes) part=1 off=7
[enviado] {'temperatura': 40.94, 'humedad': 63, 'direccion_viento': 'NO'} (3 bytes) part=1 off=8
[enviado] {'temperatura': 50.15, 'humedad': 70, 'direccion_viento': 'NO'} (3 bytes) part=1 off=9
[enviado] {'temperatura': 62.13, 'humedad': 51, 'direccion_viento': 'N'} (3 bytes) part=1 off=10
[enviado] {'temperatura': 63.04, 'humedad': 33, 'direccion_viento': 'SE'} (3 bytes) part=1 off=11
[enviado] {'temperatura': 43.77, 'humedad': 43, 'direccion_viento': 'O'} (3 bytes) part=1 off=12
[enviado] {'temperatura': 65.01, 'humedad': 98, 'direccion_viento': 'NE'} (3 bytes) part=1 off=13
[enviado] {'temperatura': 47.63, 'humedad': 51, 'direccion_viento': 'O'} (3 bytes) part=1 off=14
[enviado] {'temperatura': 58.67, 'humedad': 44, 'direccion_viento': 'SE'} (3 bytes) part=1 off=15
```

Consumidor (python consumer.py --modo compact)

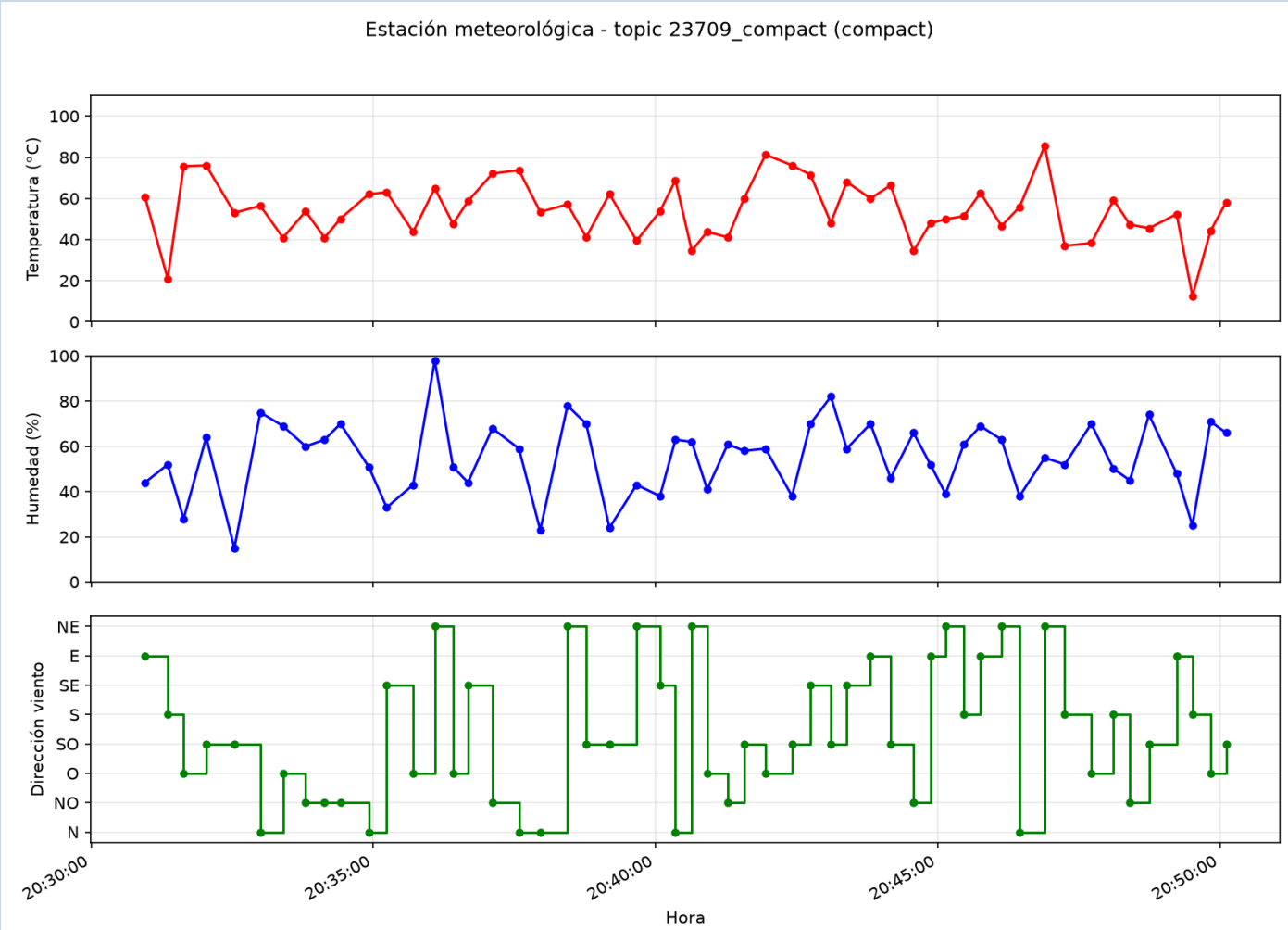
```
(.venv) gonzalomonzonch@gonzalos-ubuntu:~/Desktop/IOT_Lab$ python consumer.py --modo compact
Consumidor listo -> topic=23709_compact modo=compact
[recibido] {'temperatura': 60.66, 'humedad': 44, 'direccion_viento': 'E'}
[recibido] {'temperatura': 21.02, 'humedad': 52, 'direccion_viento': 'S'}
[recibido] {'temperatura': 75.73, 'humedad': 28, 'direccion_viento': 'O'}
[recibido] {'temperatura': 76.07, 'humedad': 64, 'direccion_viento': 'SO'}
[recibido] {'temperatura': 53.03, 'humedad': 15, 'direccion_viento': 'SO'}
[recibido] {'temperatura': 56.43, 'humedad': 75, 'direccion_viento': 'N'}
[recibido] {'temperatura': 40.92, 'humedad': 69, 'direccion_viento': 'O'}
[recibido] {'temperatura': 53.77, 'humedad': 60, 'direccion_viento': 'NO'}
[recibido] {'temperatura': 40.94, 'humedad': 63, 'direccion_viento': 'NO'}
[recibido] {'temperatura': 50.15, 'humedad': 70, 'direccion_viento': 'NO'}
[recibido] {'temperatura': 62.13, 'humedad': 51, 'direccion_viento': 'N'}
[recibido] {'temperatura': 63.04, 'humedad': 33, 'direccion_viento': 'SE'}
[recibido] {'temperatura': 43.77, 'humedad': 43, 'direccion_viento': 'O'}
[recibido] {'temperatura': 65.01, 'humedad': 98, 'direccion_viento': 'NE'}
[recibido] {'temperatura': 47.63, 'humedad': 51, 'direccion_viento': 'O'}
```

3 bytes por mensaje
Cada envío indica (3 bytes), contra 62–63 bytes en JSON.

Offsets 0 al 15
El productor envió 16 lecturas, todas en la partición 1.

Decodificación correcta
Los valores del consumidor coinciden con los del productor.

Gráfica en tiempo real (3 bytes)



Mismo resultado
Envío, consumo y despliegue gráfico funcionan igual que con JSON. Captura tras unos 20 minutos de ejecución.

Decodificación en el consumidor
Cada mensaje de 3 bytes se convierte de nuevo en temperatura, humedad y viento antes de graficar.

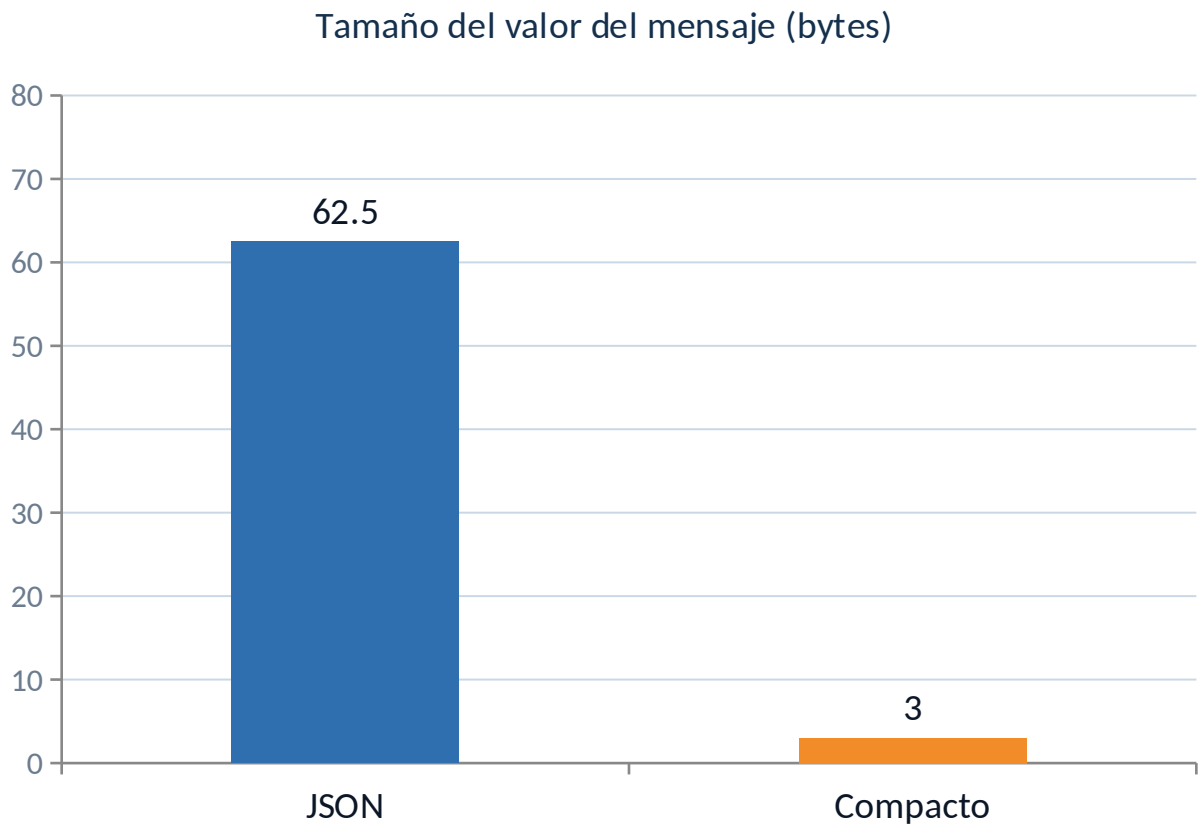
Misma resolución
La temperatura conserva sus 2 decimales gracias al entero en centésimas.

JSON vs 3 bytes

62–63 bytes por mensaje en JSON

3 bytes por mensaje en formato compacto

≈ 95 % menos bytes en el valor del mensaje



Se mide el valor (value) del mensaje, que es el payload. La key sensor1 y los encabezados de Kafka no se cuentan. JSON promedia 62.5 bytes en las capturas.

Payload restringido: complejidades

¿Qué complejidades introduce un payload pequeño?

- Hay que diseñar y documentar un formato binario, y productor y consumidor deben coincidir exactamente
- Se pierde lo autodescriptivo: sin nombres de campo, un bit mal leído da datos incorrectos
- Cada dato se cuantiza: rango y precisión quedan fijos
- Es difícil de extender: un sensor nuevo obliga a rediseñar los bits
- Es más difícil de depurar y no trae validación (sin checksum ni marca de tiempo)

¿Cómo cabe la temperatura en 14 bits?

Un float ocupa 4 bytes (32 bits), por eso no cabe. Pero la temperatura va de 0 a 110.00 con 2 decimales.

Se multiplica por 100 y se guarda como entero, de 0 a 11000. Como 2^14 = 16384 > 11000, entra en 14 bits.

Al decodificar se divide entre 100 y se recuperan los 2 decimales sin pérdida.

```
56.32 × 100 = 5632  →  14 bits
```

Humedad float y Kafka con límites

¿Y si la humedad fuera float con 1 decimal?

Iría de 0.0 a 100.0, o sea 1001 valores: se necesitan 10 bits (2^10 = 1024). El total sería 14 + 10 + 3 = 27 bits, y no caben en 24.

Decisiones posibles:

- Bajar la temperatura a 1 decimal: 0–1100 usa 11 bits, y 11 + 10 + 3 = 24
- Ampliar el payload a 4 bytes si la red lo permite
- Guardar la humedad en pasos de 0.5 o enviarla menos seguido

¿Qué ayudaría de Kafka con límites más fuertes?

- Compresión (compression.type: gzip, snappy, lz4, zstd): rinde cuando hay lotes
- Batching (batch.size, linger.ms): varias lecturas en un solo envío
- Serialización binaria con esquema (Avro o Protobuf con Schema Registry) en vez de JSON
- Key corta u omitida, y límite de tamaño con message.max.bytes
- Un gateway en el borde que reciba el payload mínimo y lo publique en Kafka

Conclusiones

Kafka desacopló al sensor de la gráfica

El patrón Pub/Sub permitió leer el historial del topic y graficar en vivo.

3 bytes bastan para el mismo flujo

Se envía, consume y grafica con ≈ 95 % menos bytes por mensaje.

Cada bit cuenta

Diseñar el payload exige conocer el rango y la resolución de cada dato.

Repositorio: github.com/djuarez-2017510/IOT_Lab · Daniel Juárez 23709 · Alexander de la Cruz 23735